

Multi-Colinearity

Kachun Huang

2025-04-09

Import Library

```
library(corrplot)
```

```
## Warning: package 'corrplot' was built under R version 4.3.3
```

```
## corrplot 0.95 loaded
```

```
library(car)
```

```
## Warning: package 'car' was built under R version 4.3.3
```

```
## Loading required package: carData
```

```
library(tidyverse)
```

```
## Warning: package 'purrr' was built under R version 4.3.3
```

```
## Warning: package 'lubridate' was built under R version 4.3.3
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v readr      2.1.5
```

```
## v forcats    1.0.0      v stringr    1.5.1
```

```
## v ggplot2    3.5.1      v tibble     3.2.1
```

```
## v lubridate  1.9.4      v tidyr      1.3.1
```

```
## v purrr      1.0.4
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## x dplyr::recode()  masks car::recode()
```

```
## x purrr::some()    masks car::some()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.3.3
```

```
## Loading required package: lattice

## Warning: package 'lattice' was built under R version 4.3.3

##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```
library(glmnet)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
##
## Loaded glmnet 4.1-8
```

Load Data

```
data <- read.csv("/Users/thomashuang/Downloads/data_factorized.csv")
```

```
str(data)
```

```
## 'data.frame':   400 obs. of  26 variables:
## $ X      : int  1 2 3 4 5 6 7 8 9 10 ...
## $ age    : num  -0.2455 -2.0793 0.6398 -0.2455 -0.0558 ...
## $ bp     : num   0.37 -2.519 0.37 -0.453 0.37 ...
## $ sg     : num   1.02 1.02 1.01 1 1.01 ...
## $ al     : int   1 4 2 4 2 3 0 2 3 2 ...
## $ su     : int   0 0 3 0 0 0 0 4 0 0 ...
## $ rbc    : chr   "normal" "normal" "normal" "normal" ...
## $ pc     : chr   "normal" "normal" "normal" "abnormal" ...
## $ pcc    : chr   "notpresent" "notpresent" "notpresent" "present" ...
## $ ba     : chr   "notpresent" "notpresent" "notpresent" "notpresent" ...
## $ bgr    : num  -0.226 -0.679 2.761 -0.306 -0.541 ...
## $ bu     : num  -0.305 -1.336 0.28 0.363 -0.792 ...
## $ sc     : num  -0.456 -0.781 -0.067 0.803 -0.316 ...
## $ sod    : num   0.236 0.236 -1.091 -2.725 1.257 ...
## $ pot    : num  -0.4579 0.607 0.4822 -2.4288 -0.0398 ...
## $ hemo   : num   1.022 -0.5 -1.131 -0.537 -0.388 ...
## $ pcv    : num   0.619 -0.104 -0.947 -0.826 -0.465 ...
## $ wbcc   : num  -0.0393 -0.8897 -0.1664 -0.532 -0.254 ...
## $ rbcc   : num   0.62894 -0.40977 0.21346 -0.72139 0.00571 ...
## $ htn    : chr   "yes" "no" "no" "yes" ...
```

```
## $ dm : chr "yes" "no" "yes" "no" ...
## $ cad : chr "no" "no" "no" "no" ...
## $ appet: chr "good" "good" "poor" "poor" ...
## $ pe : chr "no" "no" "no" "yes" ...
## $ ane : chr "no" "no" "yes" "yes" ...
## $ class: int 1 1 1 1 1 1 1 1 1 1 ...
```

Exclude Index Column

```
data <- subset(data, select = -c(X))
```

Convert Column Type

```
# Define the column names as a character vector
numeric_cols <- c("age", "bp", "bgr", "bu", "sc", "sod", "pot", "hemo", "pcv", "wbcc", "rbcc")

# Convert each column to numeric
for (col in numeric_cols) {
  data[[col]] <- as.numeric(data[[col]])
}

# Define the column names as a character vector
categorical_cols <- c("sg", "al", "su", "rbc", "pc", "pcc", "ba", "htn", "dm", "cad", "appet", "pe", "ane")

# Convert each column to factor
for (col in categorical_cols) {
  data[[col]] <- as.factor(data[[col]])
}
```

```
str(data)
```

```
## 'data.frame': 400 obs. of 25 variables:
## $ age : num -0.2455 -2.0793 0.6398 -0.2455 -0.0558 ...
## $ bp : num 0.37 -2.519 0.37 -0.453 0.37 ...
## $ sg : Factor w/ 5 levels "1.005","1.01",...: 4 4 2 1 2 3 2 3 3 4 ...
## $ al : Factor w/ 6 levels "0","1","2","3",...: 2 5 3 5 3 4 1 3 4 3 ...
## $ su : Factor w/ 6 levels "0","1","2","3",...: 1 1 4 1 1 1 1 5 1 1 ...
## $ rbc : Factor w/ 2 levels "abnormal","normal": 2 2 2 2 2 2 2 2 2 1 ...
## $ pc : Factor w/ 2 levels "abnormal","normal": 2 2 2 1 2 2 2 1 1 1 ...
## $ pcc : Factor w/ 2 levels "notpresent","present": 1 1 1 2 1 1 1 1 2 2 ...
## $ ba : Factor w/ 2 levels "notpresent","present": 1 1 1 1 1 1 1 1 1 1 ...
## $ bgr : num -0.226 -0.679 2.761 -0.306 -0.541 ...
## $ bu : num -0.305 -1.336 0.28 0.363 -0.792 ...
## $ sc : num -0.456 -0.781 -0.067 0.803 -0.316 ...
## $ sod : num 0.236 0.236 -1.091 -2.725 1.257 ...
## $ pot : num -0.4579 0.607 0.4822 -2.4288 -0.0398 ...
## $ hemo : num 1.022 -0.5 -1.131 -0.537 -0.388 ...
## $ pcv : num 0.619 -0.104 -0.947 -0.826 -0.465 ...
## $ wbcc : num -0.0393 -0.8897 -0.1664 -0.532 -0.254 ...
```

```
## $ rbcc : num 0.62894 -0.40977 0.21346 -0.72139 0.00571 ...
## $ htn : Factor w/ 2 levels "no","yes": 2 1 1 2 1 2 1 1 2 2 ...
## $ dm : Factor w/ 2 levels "no","yes": 2 1 2 1 1 2 1 2 2 2 ...
## $ cad : Factor w/ 2 levels "no","yes": 1 1 1 1 1 1 1 1 1 1 ...
## $ appet: Factor w/ 2 levels "good","poor": 1 1 2 2 1 1 1 1 1 2 ...
## $ pe : Factor w/ 2 levels "no","yes": 1 1 1 2 1 2 1 2 1 1 ...
## $ ane : Factor w/ 2 levels "no","yes": 1 1 2 2 1 1 1 1 2 2 ...
## $ class: Factor w/ 2 levels "0","1": 2 2 2 2 2 2 2 2 2 2 ...
```

Correlation Analysis

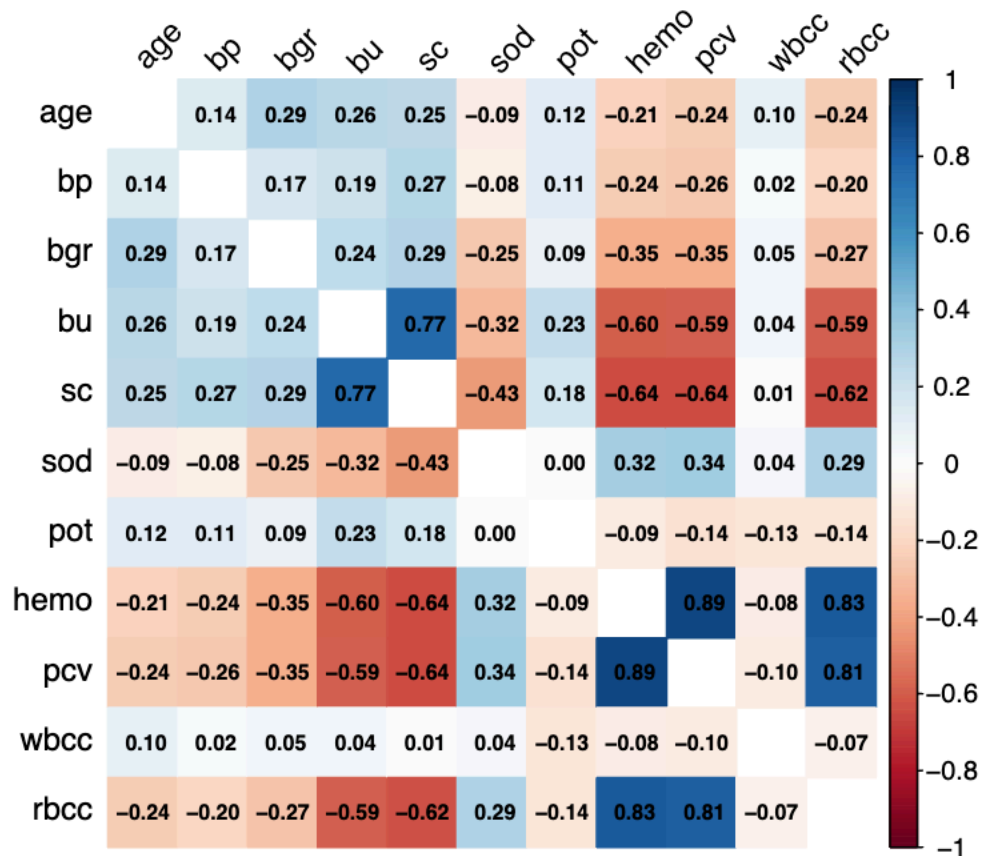
Correlation Matrix

```
cor_matrix<-cor(
  subset(data, select = -c(sg, al, su, rbc, pc, pcc, ba, htn, dm, cad, appet, pe, ane, class))
)
cor_matrix
```

```
##          age          bp          bgr          bu          sc          sod
## age  1.00000000  0.13906657  0.29477430  0.26195747  0.251120102 -0.089584246
## bp   0.13906657  1.00000000  0.16728752  0.19127260  0.272693301 -0.075950078
## bgr  0.29477430  0.16728752  1.00000000  0.24428579  0.285676596 -0.254904993
## bu   0.26195747  0.19127260  0.24428579  1.00000000  0.769836819 -0.319394059
## sc   0.25112010  0.27269330  0.28567660  0.76983682  1.000000000 -0.426248504
## sod  -0.08958425 -0.07595008 -0.25490499 -0.31939406 -0.426248504  1.000000000
## pot  0.11851605  0.11148016  0.08784174  0.23076362  0.179275046  0.001025874
## hemo -0.21266743 -0.24304107 -0.35457406 -0.59704148 -0.644561110  0.323916928
## pcv  -0.24115989 -0.25970572 -0.35420520 -0.58877742 -0.642215003  0.337440594
## wbcc  0.10165981  0.02205879  0.04850876  0.04442923  0.005275735  0.038973357
## rbcc -0.24244793 -0.20285836 -0.27158282 -0.58802414 -0.623465048  0.293824393
##          pot          hemo          pcv          wbcc          rbcc
## age  0.118516046 -0.21266743 -0.24115989  0.101659810 -0.24244793
## bp   0.111480164 -0.24304107 -0.25970572  0.022058787 -0.20285836
## bgr  0.087841740 -0.35457406 -0.35420520  0.048508764 -0.27158282
## bu   0.230763624 -0.59704148 -0.58877742  0.044429228 -0.58802414
## sc   0.179275046 -0.64456111 -0.64221500  0.005275735 -0.62346505
## sod  0.001025874  0.32391693  0.33744059  0.038973357  0.29382439
## pot  1.000000000 -0.09492757 -0.14126549 -0.132130343 -0.13642304
## hemo -0.094927575  1.00000000  0.89237703 -0.081455308  0.83054176
## pcv  -0.141265488  0.89237703  1.00000000 -0.099980849  0.81179317
## wbcc -0.132130343 -0.08145531 -0.09998085  1.000000000 -0.06960186
## rbcc -0.136423041  0.83054176  0.81179317 -0.069601859  1.000000000
```

Heat Map

```
corrplot(cor_matrix, method = "color", addCoef.col = "black", # Add values in black
  tl.col = "black", tl.srt = 45, # Tilted labels
  diag = FALSE, # Hide diagonal if desired
  number.cex = 0.7 # Adjust text size
)
```



- pcv vs. hemo **0.9**
- hemo vs. rbcc **0.8**
- rbcc vs. pcv **0.79**

These 3 combo all have really high correlation.

Explain

Scientifically speaking, they are all directly related to each other.

- pcv vs. hemo
 - The more red blood cells you have (i.e. higher PCV), the more hemoglobin you're likely to have — they rise and fall together.
- hemo vs. rbcc
 - More red blood cells, more hemoglobin total (assuming each cell has roughly the same amount). Again.
- rbcc vs. pcv
 - Higher RBC count means a greater proportion of blood is occupied by these cells, higher PCV.

These values suggest moderate multicollinearity between PCV and hemoglobin.

Penalized Regression

Prepare Data

```
# Prepare data
X <- as.matrix(data %>% select(-class))
y <- data$class

# Split into training and test sets
set.seed(597)
training.samples <- y %>% createDataPartition(p = 0.7, list = FALSE)
X_train <- X[training.samples, ]
X_test <- X[-training.samples, ]
y_train <- y[training.samples]
y_test <- y[-training.samples]
train.data <- data[training.samples,]
test.data <- data[-training.samples,]
```

Ridge Regression

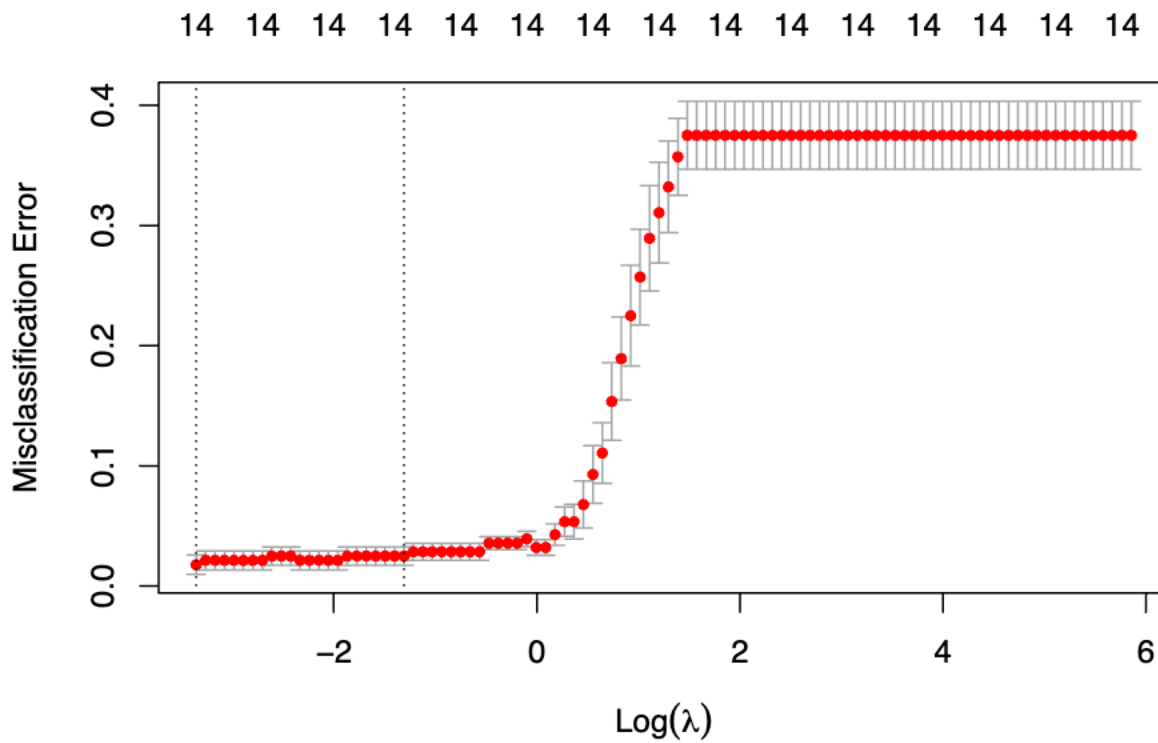
```
cv_ridge <- cv.glmnet(
  x = X_train,
  y = y_train,
  family = "binomial",
  alpha = 0,           # Ridge regression
  nfolds = 10,         # 10-fold cross-validation
  type.measure = "class" # Use misclassification error
)
```

```
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
```

```
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
```

CV plot

```
plot(cv_ride)
```



Best Lambda

```
best_lambda <- cv_ride$lambda.min  
cat("Optimal lambda:", best_lambda, "\n")
```

```
## Optimal lambda: 0.03488839
```

Model with best lambda

```
ridge_model <- glmnet(  
  x = X_train,  
  y = y_train,  
  family = "binomial",  
  alpha = 0,  
  lambda = best_lambda  
)
```

```
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
```

```
coef(ridge_model)
```

```
## 25 x 1 sparse Matrix of class "dgCMatrix"
##              s0
## (Intercept) 202.36074341
## age         0.06059819
## bp          0.24419872
## sg         -197.61052975
## al          0.60799903
## su          0.30763314
## rbc         .
## pc          .
## pcc         .
## ba          .
## bgr         0.27478595
## bu          0.14871149
## sc          0.69192345
## sod        -0.27633358
## pot        -0.09702338
## hemo       -0.74524303
## pcv        -0.59978325
## wbcc       0.02691589
## rbcc      -0.46130322
## htn        .
## dm         .
## cad         .
## appet       .
## pe         .
## ane        .
```

Prediction

```
pred_prob <- predict(ridge_model,
                     newx = X_test,
                     type = "response")
```

```
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
```

```
pred_class <- ifelse(pred_prob > 0.5, 1, 0)
```

Confusion Matrix

```
cm_ride <- confusionMatrix(factor(pred_class), y_test, positive = "1")
print(cm_ride)
```

```
## Confusion Matrix and Statistics
##
##              Reference
```

```
## Prediction 0 1
##           0 45 5
##           1 0 70
##
##           Accuracy : 0.9583
##           95% CI : (0.9054, 0.9863)
##           No Information Rate : 0.625
##           P-Value [Acc > NIR] : < 2e-16
##
##           Kappa : 0.913
##
## Mcnemar's Test P-Value : 0.07364
##
##           Sensitivity : 0.9333
##           Specificity : 1.0000
##           Pos Pred Value : 1.0000
##           Neg Pred Value : 0.9000
##           Prevalence : 0.6250
##           Detection Rate : 0.5833
##           Detection Prevalence : 0.5833
##           Balanced Accuracy : 0.9667
##
##           'Positive' Class : 1
##
```

Lasso

```
cv_lasso <- cv.glmnet(
  x = X_train,
  y = y_train,
  family = "binomial",
  alpha = 1,           # Lasso regression
  nfolds = 10,         # 10-fold cross-validation
  type.measure = "class" # Use misclassification error
)
```

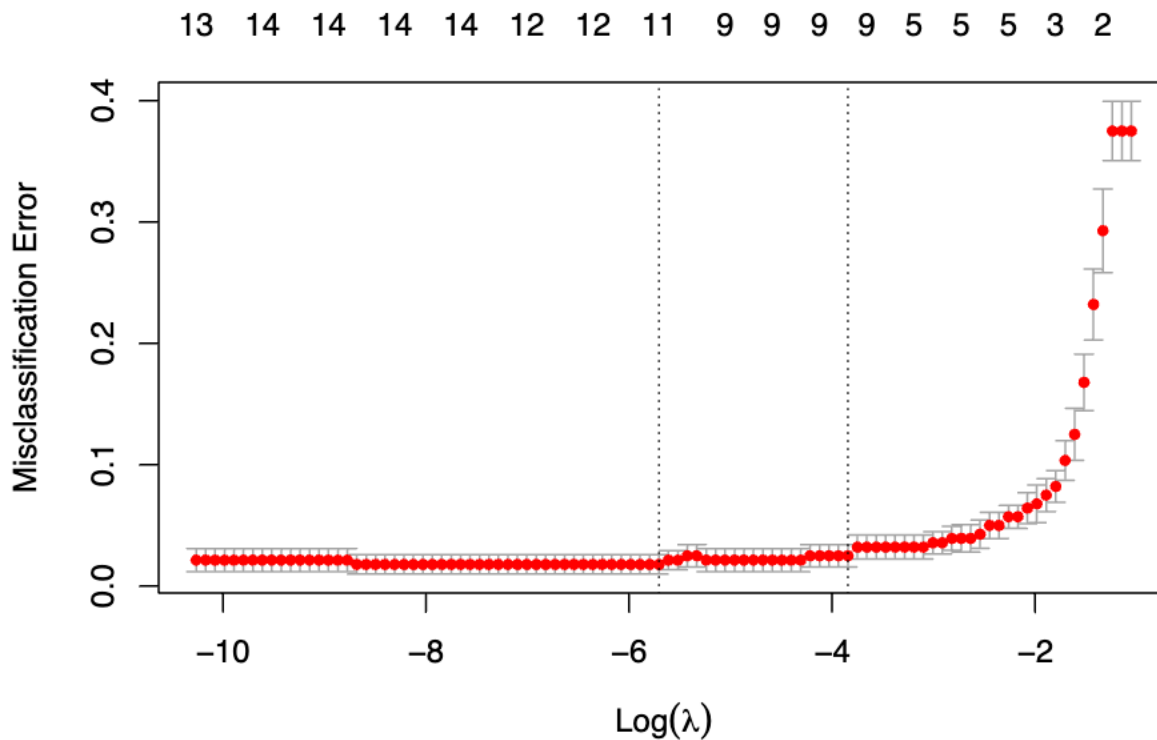
```
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion

## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
```

```
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
```

CV plot

```
plot(cv_lasso)
```



Best Lambda

```
best_lambda <- cv_lasso$lambda.min
cat("Optimal lambda:", best_lambda, "\n")
```

```
## Optimal lambda: 0.003330266
```

Model with best lambda

```
lasso_model <- glmnet(
  x = X_train,
  y = y_train,
  family = "binomial",
```

```

alpha = 1,
lambda = best_lambda
)

```

```
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
```

```
coef(lasso_model)
```

```

## 25 x 1 sparse Matrix of class "dgCMatrix"
##              s0
## (Intercept) 637.7260874
## age          .
## bp           0.2742011
## sg          -623.2708848
## al           1.9218059
## su           0.6355298
## rbc          .
## pc           .
## pcc          .
## ba           .
## bgr          0.2137736
## bu           0.1277687
## sc           3.2672412
## sod          .
## pot         -0.2260748
## hemo         -1.6057453
## pcv         -1.1585919
## wbcc         .
## rbcc        -0.1866137
## htn          .
## dm           .
## cad          .
## appet        .
## pe           .
## ane          .

```

Prediction

```

pred_prob <- predict(lasso_model,
                     newx = X_test,
                     type = "response")

```

```
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
```

```
pred_class <- ifelse(pred_prob > 0.5, 1, 0)
```

Confusion Matrix

```
cm_lasso <- confusionMatrix(factor(pred_class), y_test, positive = "1")
print(cm_lasso)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##           0 45  2
##           1  0 73
##
##           Accuracy : 0.9833
##           95% CI : (0.9411, 0.998)
##           No Information Rate : 0.625
##           P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.9648
##
## Mcnemar's Test P-Value : 0.4795
##
##           Sensitivity : 0.9733
##           Specificity : 1.0000
##           Pos Pred Value : 1.0000
##           Neg Pred Value : 0.9574
##           Prevalence : 0.6250
##           Detection Rate : 0.6083
##           Detection Prevalence : 0.6083
##           Balanced Accuracy : 0.9867
##
##           'Positive' Class : 1
##
```

Elastic Net

```
# Define cross-validation control
ctrl <- trainControl(
  method = "cv",           # Cross-validation
  number = 10,             # 10 folds
  classProbs = TRUE,       # For probability predictions
  summaryFunction = twoClassSummary, # For ROC/AUC metrics
  verboseIter = TRUE       # Show progress
)

# response need to be name
train.data$class <- ifelse(train.data$class == "1", "ckd", "notckd")

set.seed(123) # For reproducibility
model <- train(
  class~,                 # Predict class using all predictors
  data = train.data,
  method = "glmnet",
  trControl = ctrl,
```

```
metric = "ROC"           # Optimize for AUC
)
```

```
## + Fold01: alpha=0.10, lambda=0.06978
## - Fold01: alpha=0.10, lambda=0.06978
## + Fold01: alpha=0.55, lambda=0.06978
## - Fold01: alpha=0.55, lambda=0.06978
## + Fold01: alpha=1.00, lambda=0.06978
## - Fold01: alpha=1.00, lambda=0.06978
## + Fold02: alpha=0.10, lambda=0.06978
## - Fold02: alpha=0.10, lambda=0.06978
## + Fold02: alpha=0.55, lambda=0.06978
## - Fold02: alpha=0.55, lambda=0.06978
## + Fold02: alpha=1.00, lambda=0.06978
## - Fold02: alpha=1.00, lambda=0.06978
## + Fold03: alpha=0.10, lambda=0.06978
## - Fold03: alpha=0.10, lambda=0.06978
## + Fold03: alpha=0.55, lambda=0.06978
## - Fold03: alpha=0.55, lambda=0.06978
## + Fold03: alpha=1.00, lambda=0.06978
## - Fold03: alpha=1.00, lambda=0.06978
## + Fold04: alpha=0.10, lambda=0.06978
## - Fold04: alpha=0.10, lambda=0.06978
## + Fold04: alpha=0.55, lambda=0.06978
## - Fold04: alpha=0.55, lambda=0.06978
## + Fold04: alpha=1.00, lambda=0.06978
## - Fold04: alpha=1.00, lambda=0.06978
## + Fold05: alpha=0.10, lambda=0.06978
## - Fold05: alpha=0.10, lambda=0.06978
## + Fold05: alpha=0.55, lambda=0.06978
## - Fold05: alpha=0.55, lambda=0.06978
## + Fold05: alpha=1.00, lambda=0.06978
## - Fold05: alpha=1.00, lambda=0.06978
## + Fold06: alpha=0.10, lambda=0.06978
## - Fold06: alpha=0.10, lambda=0.06978
## + Fold06: alpha=0.55, lambda=0.06978
## - Fold06: alpha=0.55, lambda=0.06978
## + Fold06: alpha=1.00, lambda=0.06978
## - Fold06: alpha=1.00, lambda=0.06978
## + Fold07: alpha=0.10, lambda=0.06978
## - Fold07: alpha=0.10, lambda=0.06978
## + Fold07: alpha=0.55, lambda=0.06978
## - Fold07: alpha=0.55, lambda=0.06978
## + Fold07: alpha=1.00, lambda=0.06978
## - Fold07: alpha=1.00, lambda=0.06978
## + Fold08: alpha=0.10, lambda=0.06978
## - Fold08: alpha=0.10, lambda=0.06978
## + Fold08: alpha=0.55, lambda=0.06978
## - Fold08: alpha=0.55, lambda=0.06978
## + Fold08: alpha=1.00, lambda=0.06978
## - Fold08: alpha=1.00, lambda=0.06978
## + Fold09: alpha=0.10, lambda=0.06978
## - Fold09: alpha=0.10, lambda=0.06978
```

```
## + Fold09: alpha=0.55, lambda=0.06978
## - Fold09: alpha=0.55, lambda=0.06978
## + Fold09: alpha=1.00, lambda=0.06978
## - Fold09: alpha=1.00, lambda=0.06978
## + Fold10: alpha=0.10, lambda=0.06978
## - Fold10: alpha=0.10, lambda=0.06978
## + Fold10: alpha=0.55, lambda=0.06978
## - Fold10: alpha=0.55, lambda=0.06978
## + Fold10: alpha=1.00, lambda=0.06978
## - Fold10: alpha=1.00, lambda=0.06978
## Aggregating results
## Selecting tuning parameters
## Fitting alpha = 0.1, lambda = 0.0698 on full training set
```

Best Tune

```
model$bestTune
```

```
##      alpha      lambda
## 3    0.1 0.06977677
```

Model with bestTune

```
model_en <- glmnet(
  x = X_train,
  y = y_train,
  alpha=model$bestTune$alpha,
  lambda=model$bestTune$lambda,
  family = "binomial"
)
```

```
## Warning in storage.mode(xd) <- "double": NAs introduced by coercion
```

```
coef(model_en)
```

```
## 25 x 1 sparse Matrix of class "dgCMatrix"
##              s0
## (Intercept) 153.34626017
## age         0.04371002
## bp          0.15868332
## sg         -149.76894411
## al          0.43777379
## su          0.19165955
## rbc         .
## pc          .
## pcc         .
## ba          .
## bgr         0.20890348
## bu         0.09153812
```

```
## sc          0.50104038
## sod         -0.17693866
## pot         -0.01836818
## hemo        -0.59441873
## pcv         -0.48495595
## wbcc        .
## rbcc        -0.38169159
## htn         .
## dm          .
## cad         .
## appet       .
## pe          .
## ane         .
```

Prediction

```
pred_prob <- predict(model_en,
                      newx = X_test,
                      type = "response")
```

```
## Warning in cbind2(1, newx) %*% nbeta: NAs introduced by coercion
```

```
pred_class <- ifelse(pred_prob > 0.5, 1, 0)
```

Confusion Matrix

```
cm_en <- confusionMatrix(factor(pred_class), y_test, positive = "1")
print(cm_en)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction  0  1
##           0 45  6
##           1  0 69
##
##              Accuracy : 0.95
##              95% CI : (0.8943, 0.9814)
##      No Information Rate : 0.625
##      P-Value [Acc > NIR] : < 2e-16
##
##              Kappa : 0.8961
##
##  McNemar's Test P-Value : 0.04123
##
##              Sensitivity : 0.9200
##              Specificity : 1.0000
##      Pos Pred Value : 1.0000
##      Neg Pred Value : 0.8824
```

```
##           Prevalence : 0.6250
##       Detection Rate : 0.5750
## Detection Prevalence : 0.5750
##       Balanced Accuracy : 0.9600
##
##       'Positive' Class : 1
##
```

Dimensionality Reduction

SVD

PCA